

UNIVERSIDAD CONTINENTAL

FACULTAD DE INGENIERÍA
ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

INFORME TÉCNICO DE EXAMEN PARCIAL ESTRUCTURA DE DATOS

CASO PRÁCTICO: GESTIÓN DE UNA FLOTA DE VEHÍCULOS ELÉCTRICOS
Tipo D - Evaluación Individual

ASIGNATURA:	Estructura de Datos
DOCENTE:	Dr. Ing. Julio Arboleda H.
SECCIÓN:	24UC00428
FECHA DE ENTREGA:	05 de Octubre de 2026
ESCALA DE CALIFICACIÓN:	0 a 20 puntos (12 pts Código + 8 pts Informe)

1. Introducción

1.1. Descripción del Sistema

En el contexto actual de transición energética y modernización del transporte urbano, la gestión operativa de flotas de vehículos 100% eléctricos (EV) plantea desafíos logísticos específicos. A diferencia de los vehículos de combustión interna tradicionales, las unidades eléctricas dependen de factores críticos como la capacidad neta de almacenamiento en sus baterías (kWh) y la tasa de eficiencia o consumo promedio (kWh/100km). El presente sistema ha sido desarrollado para satisfacer las necesidades operativas de una empresa de alquiler de vehículos eléctricos, proporcionando una solución integral en consola estructurada bajo el estándar C++.

1.2. Objetivo del Programa

El objetivo primordial del software es centralizar y automatizar el ciclo de vida operativo de cada unidad de la flota. Esto comprende el registro controlado de vehículos evitando colisiones de identificadores, la actualización dinámica del historial de viajes, la conmutación de estados operativos (Disponible vs. En mantenimiento), el cómputo en tiempo real de la autonomía estimada y la generación de reportes estratégicos para la toma de decisiones, tales como la detección temprana de unidades con baja autonomía y la identificación de vehículos sometidos a alta demanda de uso.

1.3. Justificación del Uso de Estructuras (struct) y Arreglos Unidimensionales

Para modelar el dominio del problema se seleccionaron dos conceptos fundamentales de la programación estructurada:

- **Registros (struct):** Un vehículo eléctrico es una entidad compleja compuesta por atributos heterogéneos (identificador numérico entero, modelo alfanumérico, capacidad de batería real, consumo promedio real, contador de viajes entero y estado operativo). El uso de 'struct Vehiculo' permite encapsular todas estas propiedades bajo un mismo tipo de dato definido por el usuario, garantizando cohesión conceptual, paso de parámetros por referencia limpia y evitando el uso ineficiente y propenso a errores de arreglos paralelos desarticulados.
- **Arreglos Unidimensionales (Vectores Estáticos):** La colección de vehículos se almacena en un arreglo unidimensional estático continuo en memoria ('Vehiculo flota[MAX_VEHICULOS]'). Esta elección proporciona acceso aleatorio inmediato de orden temporal constante $O(1)$ mediante indexación directa, predictibilidad en el consumo de memoria en la pila (stack) sin sobrecarga (overhead) de punteros dinámicos, y constituye la estructura ideal para la aplicación directa de algoritmos de ordenamiento iterativo (burbuja) y búsquedas lineales solicitadas.

FUNDAMENTO ARQUITECTÓNICO: La combinación de registros (struct) y arreglos unidimensionales estáticos modela fielmente las arquitecturas de sistemas embebidos y aplicaciones de control industrial, donde los recursos de memoria son finitos y se prioriza la simplicidad, predictibilidad y rendimiento determinístico.

2. Lógica del Ingreso de Datos

2.1. Flujo de Captura de Datos

El flujo de captura de datos sigue una secuencia rigurosa que impide que un registro corrupto o inconsistente ingrese al estado del programa. El proceso se ejecuta en los siguientes pasos:

- **1. Verificación de capacidad:** Se valida preliminarmente que el contador 'totalVehiculos' sea estrictamente menor que 'MAX_VEHICULOS'.
- **2. Captura y verificación de ID:** Se solicita el identificador numérico y se verifica inmediatamente su inexistencia en el arreglo mediante búsqueda secuencial.
- **3. Captura del modelo:** Se utiliza 'getline(cin, nuevo.modelo)' para permitir nombres comerciales con espacios compuestos (ej. 'Tesla Model 3', 'Hyundai Ioniq 5').
- **4. Captura de capacidad y consumo:** Se leen valores decimales float mediante funciones especializadas con validación de cota superior a cero.
- **5. Captura de viajes y estado:** Se valida que los viajes sean no negativos (≥ 0) y que el estado pertenezca estrictamente al dominio {1, 2}.

2.2. Validaciones Implementadas

- **ID Único:** Se ejecuta una llamada a 'buscarVehiculoPorId(flota, total, id)'. Si el valor retornado es distinto de -1, se rechaza la entrada y se exige un nuevo ID.
- **Capacidad y Consumo (> 0):** Tanto la batería (kWh) como el consumo (kWh/100km) deben representar magnitudes físicas positivas. Se rechazan valores ≤ 0 .
- **Viajes Realizados (≥ 0):** El odómetro o contador de servicios de una unidad no puede ser negativo. Se exige $n \geq 0$.
- **Estado Operativo (1 o 2):** Se restringe la entrada a 1 (Disponible) o 2 (En mantenimiento), rechazando cualquier otro entero.

2.3. Uso de Estructuras de Control (IF, WHILE, FOR)

La validación combina estructuras condicionales 'if-else' y bucles interactivos 'while(true)'. A diferencia de una simple advertencia condicional con 'if', el uso de bucles 'while' obliga al operador a reintentar el ingreso de manera continua hasta suministrar un valor válido, impidiendo el avance con datos erróneos.

2.4. Manejo de Errores (Ingreso de Caracteres no Numéricos)

Uno de los fallos más comunes en aplicaciones C++ de consola ocurre cuando se ingresan letras o caracteres alfanuméricos en flujos 'cin >> variable_numerica'. Esto activa el banderín de fallo 'cin.fail()', interrumpe el flujo de entrada y genera bucles infinitos descontrolados. Para mitigar completamente este riesgo, se implementó una rutina robusta de sanitización del buffer:

```
void limpiarBuffer() {
    cin.clear(); // Restablece los flags de error del flujo cin
    cin.ignore(10000, '\n'); // Descarta caracteres espurios del buffer hasta el salto de linea
}

float leerFloatPositivo(const string& mensaje) {
    float valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor) {
```

```

        limpiarBuffer();
        if (valor > 0.0f) return valor;
        cout << "    [Error] El valor debe ser estrictamente mayor que cero (> 0).\n";
    } else {
        cout << "    [Error] Entrada invalida. Ingrese un valor numerico valido.\n";
        limpiarBuffer();
    }
}
}
}

```

Fragmento: Control de excepciones de entrada por teclado y vaciado de buffer

3. Lógica de Almacenamiento

3.1. Uso del Arreglo Unidimensional para Almacenar Vehículos

La flota completa reside en la variable 'Vehiculo flota[MAX_VEHICULOS];'. Cada celda del arreglo almacena una instancia completa del registro 'Vehiculo'. Al trabajar con estructuras en arreglos unidimensionales en C++, los datos se distribuyen de forma secuencial y contigua en memoria física, lo cual maximiza la localidad espacial y el aprovechamiento de la memoria caché del procesador (Cache L1/L2) durante los recorridos y ordenamientos.

3.2. Control del Número de Elementos (Variable Contador)

Debido a que el arreglo posee una dimensión física fija de 100 elementos ('MAX_VEHICULOS = 100'), es indispensable mantener una variable de tamaño lógico: 'int totalVehiculos'. Esta variable inicia en 0 (o en 4 si se consideran los registros de prueba iniciales) y cumple tres roles determinantes:

- **Puntero de inserción:** Indica exactamente el índice de la siguiente posición libre en el arreglo ('flota[totalVehiculos] = nuevo').
- **Límite de recorrido:** Define el límite superior estricto (< total) para todos los bucles de búsqueda, cálculo, ordenamiento y reportes, evitando acceder a posiciones de memoria no inicializadas.
- **Guarda de desbordamiento:** Permite evaluar la condición 'total >= MAX_VEHICULOS' antes de iniciar cualquier operación de escritura.

3.3. Limitaciones del Enfoque Estático

El modelo de asignación estática presenta ventajas de simplicidad, pero acarrea limitaciones técnicas severas en entornos de producción:

- **Riesgo de saturación (Capacidad finita):** Si la flota crece y supera las 100 unidades, el sistema debe rechazar nuevos registros de manera forzosa, interrumpiendo la escalabilidad del negocio a menos que se modifique el código fuente y se recompile.
- **Desperdicio de memoria (Underutilization):** Si la empresa sólo opera 5 vehículos, el sistema continúa reservando en memoria el espacio de 100 registros (aproximadamente $100 * 48 \text{ bytes} = 4.8 \text{ KB}$ en la pila), lo cual en entornos reducidos representa memoria ociosa.
- **Falta de flexibilidad en tiempo de ejecución:** No es posible compactar la memoria ni solicitar bloques adicionales bajo demanda sin migrar a estructuras dinámicas (memoria heap).

4. Lógica de Actualización

4.1. Procedimiento para Actualizar el Número de Viajes

La operación de actualización de viajes ('actualizarViajes') permite modificar el historial de servicios de una unidad cuando esta regresa de un ciclo de alquiler o culmina una jornada operativa. El procedimiento no recrea la unidad, sino que modifica directamente el campo 'viajesRealizados' del registro existente por referencia en memoria.

4.2. Validaciones Previas

- **Existencia de registros:** Se verifica que la flota no esté vacía ('total == 0').
- **Localización del vehículo:** Se solicita el ID y se ejecuta la búsqueda. Si la función retorna -1, se informa que la unidad no existe y se aborta la mutación.
- **Consistencia del nuevo valor:** Se valida mediante 'leerEnteroMayorIgual' que la nueva cifra sea mayor o igual a cero (≥ 0).

4.3. Uso de Búsqueda Secuencial para Localización

Para realizar la modificación en tiempo constante una vez hallado el registro, se utiliza el índice posicional devuelto por la búsqueda secuencial: 'flota[indice].viajesRealizados = nuevosViajes;'. Al trabajar directamente sobre el arreglo pasado como parámetro, la modificación impacta de inmediato en todo el sistema.

5. Lógica de Ordenación

5.1. Algoritmo Utilizado: Método de la Burbuja (Bubble Sort)

Para dar cumplimiento estricto a las especificaciones curriculares del curso, se implementó el algoritmo clásico de ordenamiento por intercambio o Método de la Burbuja (Bubble Sort). El algoritmo efectúa pasadas sucesivas sobre el arreglo, comparando pares de elementos contiguos e intercambiándolos de posición si se encuentran en el orden incorrecto. Su complejidad temporal en el peor y caso promedio es de $O(n^2)$, con una complejidad espacial óptima de $O(1)$ in-place.

5.2. Proceso de Ordenamiento por Autonomía (Magnitud Derivada)

El criterio de ordenación solicitado es de mayor a menor (descendente) según la autonomía estimada en kilómetros. Debido a que la autonomía no se almacena como un campo estático en la estructura, el algoritmo invoca dinámicamente la función 'calcularAutonomia()' para cada par de elementos adyacentes en cada ciclo interno:

```
void ordenarVehiculosPorAutonomia(Vehiculo flota[], int total) {
    for (int i = 0; i < total - 1; i++) {
        for (int j = 0; j < total - 1 - i; j++) {
            float autoActual = calcularAutonomia(flota[j]);
            float autoSiguiente = calcularAutonomia(flota[j + 1]);

            // Criterio descendente: desplaza los valores mayores hacia el inicio
            if (autoActual < autoSiguiente) {
                Vehiculo auxiliar = flota[j];
                flota[j] = flota[j + 1];
                flota[j + 1] = auxiliar;
            }
        }
    }
}
```

```

    }
  }
}

```

Fragmento: Implementación de Bubble Sort descendente sobre cálculo derivado

5.3. Justificación Técnica de Ordenar sobre el Cálculo vs. Datos Crudos

Existen razones técnicas de peso en ingeniería de software para ordenar sobre el valor computado y no sobre un atributo prealmacenado:

- **Principio de Única Fuente de Verdad (Single Source of Truth):** Si almacenáramos un campo 'float autonomia' dentro del struct, cualquier modificación futura en la capacidad de la batería o en el consumo promedio exigiría recalcular y sincronizar dicho campo manualmente. Omitir esto generaría inconsistencias graves. Calcularlo al vuelo garantiza consistencia absoluta.
- **Normalización y Ahorro de Almacenamiento:** Evita redundancia de datos calculables matemáticamente a partir de magnitudes base.
- **Evaluación Multivariable Coherente:** Un vehículo con una batería pequeña (40 kWh) pero consumo excelente (10 kWh/100km) alcanza 400 km de autonomía, mientras que otro con batería de 60 kWh y consumo ineficiente (20 kWh/100km) solo alcanza 300 km. Ordenar únicamente por capacidad o por consumo daría una visión totalmente errónea del rendimiento de la flota.

6. Lógica de Búsqueda

6.1. Búsqueda Secuencial por Identificador Único

La búsqueda secuencial (o lineal) implementada en 'buscarVehiculoPorId' recorre uno a uno los elementos activos desde el índice 0 hasta 'total - 1'. En cada iteración se evalúa la igualdad estricta 'flota[i].id == idBuscado'. Al encontrar una coincidencia, se interrumpe de inmediato el bucle y se retorna la posición 'i'.

6.2. Análisis de Casos Posibles

- **Caso Encontrado:** El ID existe. Retorna el índice 'i' [0, total-1]. Permite consultar datos, actualizar viajes o alterar estados en tiempo $O(1)$ conociendo el índice.
- **Caso No Encontrado:** El ID no existe tras agotar los 'n' elementos. Retorna '-1', lo cual activa mensajes claros de error al usuario y previene punteros colgantes o lecturas ilegales.
- **Caso ID Duplicado:** Se neutraliza en la fase de registro llamando previamente a la búsqueda. Si la búsqueda retorna distinto de -1, se prohíbe la inserción garantizando la unicidad del ID.

6.3. Mejora Potencial: Búsqueda Binaria

Si el sistema mantuviese el arreglo permanentemente ordenado por el campo 'id' (mediante inserción ordenada o un ordenamiento previo), podría implementarse una Búsqueda Binaria (Binary Search). Esto reduciría drásticamente la complejidad computacional de $O(n)$ a $O(\log_2 n)$. Para un arreglo de 100 vehículos, mientras que la búsqueda secuencial puede requerir hasta 100 comparaciones en el peor caso, la búsqueda binaria localiza el vehículo en un máximo de 7 comparaciones ($\log_2 100 \approx 6.64$).

7. Lógica de Cálculos

7.1. Fórmula Matemática de Autonomía Estimada

La autonomía teórica de un vehículo eléctrico representa la distancia en kilómetros que puede recorrer antes de que la batería se descargue totalmente. Se calcula mediante la relación directa:

$$\text{Autonomía (km)} = (\text{Capacidad de Batería (kWh)} / \text{Consumo Promedio (kWh/100km)}) * 100$$

7.2. Uso de Acumuladores para Estadísticas de Flota

En la opción 6 ('consultarAutonomia'), el sistema implementa un acumulador de punto flotante ('float acumuladorAutonomia = 0.0f;'). Mediante un bucle iterativo que suma la autonomía de cada vehículo activo, se calcula el promedio aritmético general de la flota ('promedio = acumulador / total'). Esto ofrece a la gerencia un indicador de rendimiento global de su parque automotor.

7.3. Validación Crítica: Prevención de División por Cero

La división entre cero es una operación matemática indefinida que a nivel de hardware genera una interrupción de punto flotante o produce valores 'inf' / 'NaN' que corrompen el ordenamiento y las estadísticas. Para blindar el cálculo, la función 'calcularAutonomia' cuenta con una cláusula de guarda estricta:

```
float calcularAutonomia(const Vehiculo& v) {
    // Clausula de guarda: previene division por cero o valores negativos
    if (v.consumoPromedio <= 0.0f) {
        return 0.0f;
    }
    return (v.capacidadBateria / v.consumoPromedio) * 100.0f;
}
```

Fragmento: Prevención de división por cero en el cálculo de autonomía

8. Lógica de Reportes

8.1. Condiciones y Umbrales Dinámicos

El módulo de reportes no trabaja con valores 'cableados' (hardcoded), sino que solicita al usuario los umbrales de filtrado en tiempo de ejecución, otorgando flexibilidad operativa:

- **Reporte de Autonomía Baja:** Condición booleana: 'calcularAutonomia(flota[i]) < umbralKm'. Identifica unidades que requieren recarga prioritaria o vehículos inadecuados para viajes interurbanos.
- **Reporte de Alta Demanda:** Condición booleana: 'flota[i].viajesRealizados > umbralViajes'. Identifica unidades con desgaste mecánico acelerado para programar mantenimiento preventivo.

8.2. Filtrado de Datos y Presentación Tabular

Ambos reportes aplican un patrón de filtrado secuencial con acumulador de coincidencias ('encontrados'). Los resultados se formatean visualmente como una tabla ASCII profesional utilizando manipuladores 'std::setw', 'std::left', 'std::right', 'std::fixed' y 'std::setprecision(2)' de la biblioteca <iomanip>. Si ninguna unidad cumple la condición del umbral, se notifica claramente al usuario en lugar de presentar una tabla vacía.

9. Lógica del Menú

9.1. Implementación del Menú Principal

El menú de navegación consolida de manera organizada las 9 operaciones requeridas en las bases del examen parcial. Está encapsulado en la función 'main()' y se apoya en una estructura 'switch-case' con invocaciones modulares a funciones especializadas para cada tarea, manteniendo el principio de responsabilidad única.

9.2. Uso del Bucle 'do-while'

Se empleó un bucle 'do-while' con la condición de salida 'opcion != 9'. Esta estructura de control es la óptima para menús de usuario debido a su naturaleza post-test: garantiza que la interfaz se imprima en pantalla al menos una vez y continúe iterando sucesivamente tras cada operación hasta que el usuario decida voluntariamente abandonar el programa.

9.3. Manejo de Errores y Opciones Inválidas

- **Protección contra letras:** Se captura la opción mediante 'leerEnteroMayorIgual', lo que neutraliza caracteres inválidos antes de que alcancen el switch.
- **Cláusula default:** Si el usuario digita un entero fuera del rango 1 a 9, la cláusula 'default' del 'switch' captura el evento y muestra el mensaje '[!] Opcion invalida. Por favor seleccione una opcion entre 1 y 9.'.

10. Pruebas del Sistema (Evidencias y Resultados)

A continuación se documentan las pruebas funcionales exhaustivas ejecutadas sobre el sistema, demostrando el cumplimiento riguroso de cada uno de los requisitos del caso práctico y la estabilidad ante entradas erróneas.

10.1. Caso de Prueba 1: Registro de Vehículo con Validación de Errores

- **Objetivo:** Verificar que el sistema rechace IDs duplicados, ignore entradas alfabéticas erróneas, valide cotas numéricas y registre una unidad nueva.
- **Valores probados:** ID duplicado = 101 (Rechazado); ID letra = 'abc' (Rechazado); ID válido = 105; Capacidad errónea = -10 (Rechazada); Capacidad válida = 69.0; Consumo = 17.5; Viajes = 8; Estado = 1.
- **Resultado:** El sistema bloqueó exitosamente los intentos inválidos y registró el Volvo EX30 con una autonomía calculada de 394.29 km.

```

=====
      SISTEMA DE GESTION DE FLOTA DE VEHICULOS ELECTRICOS
=====

[1] Registrar un nuevo vehiculo
...
Seleccione una opcion [1-9]: 1

-----

      REGISTRO DE NUEVO VEHICULO
-----

Ingrese ID del vehiculo (entero positivo): 101
[Error] El ID 101 ya se encuentra registrado. Ingrese otro ID.
Ingrese ID del vehiculo (entero positivo): abc
[Error] Entrada invalida. Debe ingresar un numero entero.
Ingrese ID del vehiculo (entero positivo): 105
Ingrese Modelo del vehiculo: Volvo EX30
Ingrese Capacidad de bateria en kWh (> 0): -10
[Error] El valor debe ser estrictamente mayor que cero (> 0).
Ingrese Capacidad de bateria en kWh (> 0): 69.0
Ingrese Consumo promedio en kWh/100km (> 0): 17.5
Ingrese Numero de viajes realizados (>= 0): 8
Ingrese Estado (1 = Disponible, 2 = En mantenimiento): 1

>> [Exito] Vehiculo con ID 105 registrado correctamente.
      Autonomia estimada inicial: 394.29 km.
  
```

Figura 1: Evidencia en consola de validaciones durante el registro y rechazo de errores.

10.2. Caso de Prueba 2: Búsqueda Secuencial por ID (Éxito y Fallo)

- **Objetivo:** Comprobar la respuesta de la búsqueda secuencial frente a registros existentes y no existentes.
- **Valores probados:** ID existente = 103 (BYD Han EV); ID inexistente = 999.
- **Resultado:** Para el ID 103 recuperó la ficha completa con estado 'En mantenimiento' y autonomía de 469.23 km. Para el ID 999 emitió la advertencia correspondiente.

```

Windows PowerShell - Prueba 2: Búsqueda Secuencial por ID

-----
BUSQUEDA DE VEHICULO POR ID
-----

Ingrese el ID del vehiculo a buscar: 103

>> Vehiculo encontrado exitosamente (Posicion en arreglo: 2):
- ID : 103
- Modelo : BYD Han EV
- Capacidad bateria : 85.40 kWh
- Consumo promedio : 18.20 kWh/100km
- Viajes realizados : 40
- Estado actual : En mantenimiento
- Autonomia calculada: 469.23 km

Ingrese el ID del vehiculo a buscar: 999

[!] No se encontro ningun vehiculo con el ID 999.

```

Figura 2: Evidencia de búsqueda secuencial exitosa y manejo de registro no encontrado.

10.3. Caso de Prueba 3: Ordenación por Autonomía Estimada (Método Burbuja)

- **Objetivo:** Demostrar que el algoritmo Burbuja ordena correctamente la flota de mayor a menor en base al cálculo derivado de autonomía.
- **Vehículos analizados:** BYD Han (469.23 km), Ioniq 5 (427.06 km), Model 3 (400.00 km), Volvo EX30 (394.29 km), Leaf (242.42 km).
- **Resultado:** La tabla se desplegó con los 5 vehículos ordenados en estricto orden descendente, validando el intercambio físico de registros.

```

Windows PowerShell - Prueba 3: Ordenamiento Burbuja y Listado

-----
ORDENAR VEHICULOS POR AUTONOMIA (MAYOR A MENOR)
-----

>> [Exito] Flota ordenada exitosamente por autonomia estimada (descendente).

-----
LISTADO GENERAL DE LA FLOTA
-----

+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo | Bateria | Consumo | Viajes | Estado | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 103 | BYD Han EV | 85.4 kWh | 18.2 km | 40 | Mantenimiento | 469.23 km |
| 104 | Hyundai Ioniq 5 | 72.6 kWh | 17.0 km | 30 | Disponible | 427.06 km |
| 101 | Tesla Model 3 | 60.0 kWh | 15.0 km | 25 | Disponible | 400.00 km |
| 105 | Volvo EX30 | 69.0 kWh | 17.5 km | 8 | Disponible | 394.29 km |
| 102 | Nissan Leaf | 40.0 kWh | 16.5 km | 12 | Disponible | 242.42 km |
+-----+-----+-----+-----+-----+-----+-----+

Total de unidades registradas: 5 / 100

```

Figura 3: Evidencia del ordenamiento descendente por autonomía estimada y listado general.

10.4. Caso de Prueba 4: Generación de Reportes con Filtrado por Umbral

- **Objetivo:** Evaluar la capacidad de filtrado dinámico para baja autonomía (< umbral) y alta demanda de viajes (> umbral).
- **Umbrales probados:** Umbral autonomía = 350.0 km; Umbral viajes = 25 viajes.
- **Resultado:** Filtró con precisión el Nissan Leaf (242.42 km < 350 km) en baja autonomía, y seleccionó el BYD Han (40 viajes) y Hyundai Ioniq 5 (30 viajes) en alta demanda.

```
Windows PowerShell - Prueba 4: Generacion de Reportes por Umbral

-----
MODULO DE REPORTES DE RENDIMIENTO
-----

[1] Reporte de vehiculos con AUTONOMIA BAJA (Autonomia < umbral)
[2] Reporte de vehiculos con ALTA DEMANDA (Viajes > umbral)
Seleccione tipo de reporte [1 o 2]: 1
Ingrese umbral de autonomia minima en km: 350

>> VEHICULOS CON AUTONOMIA MENOR A 350.00 KM:
+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo          | Bateria   | Consumo   | Viajes | Estado      | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 102 | Nissan Leaf     | 40.0 kWh  | 16.5 km   | 12     | Disponible  | 242.42 km      |
+-----+-----+-----+-----+-----+-----+-----+

>> Total de vehiculos identificados con autonomia baja: 1

Seleccione tipo de reporte [1 o 2]: 2
Ingrese umbral minimo de viajes completados: 25

>> VEHICULOS CON ALTA DEMANDA (VIAJES MAYOR A 25):
+-----+-----+-----+-----+-----+-----+-----+
| ID | Modelo          | Bateria   | Consumo   | Viajes | Estado      | Autonomia Est. |
+-----+-----+-----+-----+-----+-----+-----+
| 103 | BYD Han EV      | 85.4 kWh  | 18.2 km   | 40     | Mantenimiento | 469.23 km      |
| 104 | Hyundai Ioniq 5 | 72.6 kWh  | 17.0 km   | 30     | Disponible  | 427.06 km      |
+-----+-----+-----+-----+-----+-----+-----+

>> Total de vehiculos con alta demanda: 2
```

Figura 4: Evidencia de los reportes condicionales de autonomía baja y alta demanda.

10.5. Caso de Prueba 5: Actualización de Viajes y Cambio de Estado Operativo

- **Objetivo:** Comprobar la persistencia en memoria durante la ejecución de las operaciones de actualización y conmutación de estado.
- **Acciones ejecutadas:** ID 102 (Nissan Leaf): viajes aumentados de 12 a 18. ID 103 (BYD Han): estado conmutado de [2] Mantenimiento a [1] Disponible.
- **Resultado:** Ambas mutaciones se ejecutaron sin efectos colaterales en los demás atributos de los registros.

```

Windows PowerShell - Prueba 5: Actualización y Cambio de Estado

-----
                ACTUALIZAR NUMERO DE VIAJES
-----

Ingrese el ID del vehiculo a actualizar: 102
Vehiculo seleccionado: Nissan Leaf
Cantidad actual de viajes: 12
Ingrese la nueva cantidad total de viajes (>= 0): 18

>> [Exito] Numero de viajes actualizado satisfactoriamente a 18.

-----
                CAMBIAR ESTADO DE OPERACION
-----

Ingrese ID del vehiculo a modificar: 103
Unidad seleccionada: BYD Han EV (ID: 103)
Estado actual: [2] En mantenimiento
Ingrese nuevo estado (1 = Disponible, 2 = En mantenimiento): 1

>> [Exito] Estado actualizado a: Disponible.

```

Figura 5: Evidencia de actualización de odómetro y conmutación de estado operativo.

11. Oportunidades de Mejora

11.1. Reflexión sobre las Limitaciones del Sistema Estático

El sistema desarrollado satisface plenamente los requerimientos académicos del Examen Parcial de Estructura de Datos. Sin embargo, el análisis crítico de su arquitectura evidencia limitaciones intrínsecas derivadas del almacenamiento estático y la volatilidad de la memoria RAM: al finalizar el programa, toda la información de la flota se pierde. Asimismo, el límite rígido de 100 vehículos impone una barrera de crecimiento que requiere intervención en el código fuente.

11.2. Propuestas Técnicas de Evolución

- **1. Implementación de Estructuras de Datos Dinámicas:** Reemplazar el arreglo estático por una Lista Simplemente Enlazada o Doblemente Enlazada (o en su defecto un 'std::vector' de C++). Esto permitirá reservar y liberar nodos en memoria heap mediante operadores 'new' y 'delete' en tiempo de ejecución, logrando una flota de tamaño elástico ilimitado que crece y decrece de acuerdo con la demanda real sin desperdiciar bytes.
- **2. Persistencia en Archivos de Disco (Ficheros):** Integrar la biblioteca '<fstream>' de C++ para implementar persistencia de datos en archivos planos (.txt, .csv) o binarios (.dat). De este modo, al iniciar la aplicación se cargará el estado previo de la flota automáticamente y, al salir, se guardarán los cambios realizados, evitando la pérdida de información entre sesiones de trabajo.
- **3. Integración con Motor de Base de Datos Relacional:** Para un entorno corporativo real con múltiples sucursales concurrentes, se recomienda conectar la lógica de negocio con una base de datos relacional ligera como SQLite o cliente-servidor (PostgreSQL / MySQL). Esto proporcionaría propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), indexación mediante árboles B+ y soporte para consultas transaccionales concurrentes.
- **4. Interfaz Gráfica de Usuario (GUI) y Dashboard:** Migrar la consola de comandos hacia una interfaz gráfica amigable (empleando frameworks como Qt C++ o wxWidgets, o mediante una API

REST en C++ conectada a un frontend web). Esto habilitaría paneles de control visual con gráficos de barras para la autonomía, alertas cromáticas de mantenimiento y mapas de geolocalización de las unidades.